In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
```

In [3]:
```
data = pd.read_csv("marketing_data.csv",index_col="ID")
```

In [4]:
```
import warnings
```

In [5]:
```
warnings.filterwarnings('ignore')
```

In [6]:
```
data.head()
```

Out[6]:

ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	MntFruits
1826	1970	Graduation	Divorced	$84,835.00	0	0	6/16/14	0	189	104
1	1961	Graduation	Single	$57,091.00	0	0	6/15/14	0	464	5
10476	1958	Graduation	Married	$67,267.00	0	1	5/13/14	0	134	11
1386	1967	Graduation	Together	$32,474.00	1	1	5/11/14	0	10	0
5371	1989	Graduation	Single	$21,474.00	1	0	4/8/14	0	6	16

5 rows × 27 columns

Task-1 investing proper import of Data

In [7]:
```
data.dtypes

# so Income & Dt_Customer needs to be fixed
```

Out[7]:
```
Year_Birth             int64
Education             object
Marital_Status        object
 Income               object
Kidhome                int64
Teenhome               int64
Dt_Customer           object
Recency                int64
MntWines               int64
MntFruits              int64
MntMeatProducts        int64
MntFishProducts        int64
MntSweetProducts       int64
MntGoldProds           int64
NumDealsPurchases      int64
NumWebPurchases        int64
NumCatalogPurchases    int64
NumStorePurchases      int64
NumWebVisitsMonth      int64
AcceptedCmp3           int64
AcceptedCmp4           int64
AcceptedCmp5           int64
AcceptedCmp1           int64
AcceptedCmp2           int64
Response               int64
Complain               int64
Country               object
dtype: object
```

In [8]:
```
data.columns = ['Year_Birth', 'Education', 'Marital_Status', 'Income', 'Kidhome',
       'Teenhome', 'Dt_Customer', 'Recency', 'MntWines', 'MntFruits',
       'MntMeatProducts', 'MntFishProducts', 'MntSweetProducts',
       'MntGoldProds', 'NumDealsPurchases', 'NumWebPurchases',
       'NumCatalogPurchases', 'NumStorePurchases', 'NumWebVisitsMonth',
```

```
       'AcceptedCmp3', 'AcceptedCmp4', 'AcceptedCmp5', 'AcceptedCmp1',
       'AcceptedCmp2', 'Response', 'Complain', 'Country']
```

In [9]:
```
data.Income
```

Out[9]:
```
ID
1826     $84,835.00
1        $57,091.00
10476    $67,267.00
1386     $32,474.00
5371     $21,474.00
            ...
10142    $66,476.00
5263     $31,056.00
22       $46,310.00
528      $65,819.00
4070     $94,871.00
Name: Income, Length: 2240, dtype: object
```

In [10]:
```
data.Income = data.Income.str.replace('$',"").str.replace(",","").str.replace(".00","").astype('float')
```

In [11]:
```
data.Income.isnull().sum()
# there are 24 customers with No-income information
```

Out[11]:
```
24
```

In [12]:
```
# fixing data types of Dt_Customer to date type
data.Dt_Customer = pd.to_datetime(data.Dt_Customer)
```

In [13]:
```
data.dtypes;
# now all the data types are correct
```

Task-2 cleaning of Marital Staus & Education, Imputing Income

rationalize Marital Status

In [14]:
```
data.Marital_Status.value_counts()
```

Out[14]:
```
Married     864
Together    580
Single      480
Divorced    232
Widow        77
Alone         3
YOLO          2
Absurd        2
Name: Marital_Status, dtype: int64
```

In [15]:
```
data.pivot_table(index='Marital_Status',values='Income',aggfunc='mean')
# let us drop absurd cases
```

Out[15]:

Marital_Status	Income
Absurd	72365.500000
Alone	43789.000000
Divorced	51651.267241
Married	50136.843641
Single	49158.569002
Together	51770.171030
Widow	54314.697368
YOLO	48432.000000

In [16]:
```
data = data[data.Marital_Status!='Absurd']
```

In [17]:
```
data['Marital_Status'] = data['Marital_Status'].replace({'Alone':'Single','YOLO':'Single','Together':'Mar
```

In [18]:
```
data['Marital_Status'].value_counts()
```

Out[18]:
```
Married     1444
Single       485
Divorced     232
Widow         77
Name: Marital_Status, dtype: int64
```

rationalize Education

In [19]:
```
data.Education.value_counts()
```

Out[19]:
```
Graduation    1126
PhD            486
Master         369
2n Cycle       203
Basic           54
Name: Education, dtype: int64
```

In [20]:
```
data[['Marital_Status','Education']]
```

Out[20]:

ID	Marital_Status	Education
1826	Divorced	Graduation
1	Single	Graduation
10476	Married	Graduation
1386	Married	Graduation
5371	Single	Graduation
...	...	...
10142	Divorced	PhD
5263	Married	2n Cycle
22	Divorced	Graduation
528	Married	Graduation
4070	Married	PhD

2238 rows × 2 columns

imputing income

In [21]:
```
from sklearn.preprocessing import LabelEncoder
```

In [22]:
```
#Label Martial Status
labeler = LabelEncoder().fit_transform(data.Marital_Status)
data['Marital_Status_label'] = labeler
```

In [23]:
```
#Label Education Level
labeler = LabelEncoder().fit_transform(data.Education)
data['Education_label'] = labeler
```

In [24]:
```
# creating a unique labeler for combination of marital status & education
data['marital_education_labeler'] = data.Education_label.astype('str')++data.Marital_Status_label.astype(
```

In [25]:
```
# marital status wise, education level wise mean income
grouped_avg_income = data.pivot_table(values='Income',index='marital_education_labeler',aggfunc='mean')
```

In [26]:
```
grouped_avg_income.head()
```

Out[26]:

marital_education_labeler	Income
00	49395.130435
01	44049.419118
02	53673.944444
03	51392.200000
10	9548.000000

In [27]:
```
sub_data1 = data[~data.Income.isna()]
sub_data2 = data[data.Income.isna()].drop('Income',axis=1).merge(grouped_avg_income,left_on='marital_educ
data = pd.concat([sub_data1,sub_data2])
```

In [28]:
```
data.Income.isna().sum()
# now no missing values are there
```

Out[28]:
```
0
```

Task-3 Creating variables for Total Purhchase and Total number of kids

In [29]:
```
# creating variable for total number of kids
data['Kid_total'] = data['Kidhome']+data['Teenhome']
```

In [30]:
```
# creating variable for total purchases
data['Total_purchases'] = data['NumWebPurchases']+data['NumCatalogPurchases']+data['NumStorePurchases']
```

Task-4 Histogram, Boxplot and Outlier Detection

In [31]:
```
data.describe()
```

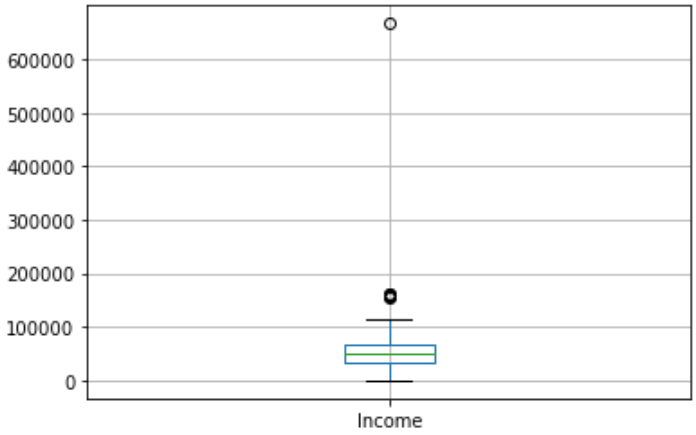
Out[31]:

	Year_Birth	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProducts	Mntl
count	2238.000000	2238.000000	2238.000000	2238.000000	2238.000000	2238.000000	2238.000000	2238.000000	
mean	1968.800268	50645.887363	0.444593	0.506702	49.105898	303.889634	26.250223	166.819929	
std	11.975906	26347.015378	0.538475	0.544571	28.974777	336.726585	39.749663	225.704653	
min	1893.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	1959.000000	34125.750000	0.000000	0.000000	24.000000	23.250000	1.000000	16.000000	
50%	1970.000000	50521.500000	0.000000	0.000000	49.000000	173.000000	8.000000	67.000000	
75%	1977.000000	67465.250000	1.000000	1.000000	74.000000	504.750000	33.000000	231.750000	
max	1996.000000	666666.000000	2.000000	2.000000	99.000000	1493.000000	199.000000	1725.000000	

8 rows × 27 columns

Income variable

In [32]:
```
data[['Income']].boxplot()
plt.show()
```



In [33]:
```
data.Income.quantile(q=(0.25,0.75,0.8,0.9,0.99)).values
# so clearly any observation beyond 1,00,000 may be excluded
# we are not aggressive in dropping 5% of the observations
```

Out[33]:
```
array([34125.75, 67465.25, 71149.  , 79430.1 , 94439.44])
```

In [34]:
```
q1,q3 = data.Income.quantile((0.25,0.75))
iqr = q3-q1
lower = q1-1.5*iqr
upper = q3+1.5*iqr
lower,upper
```

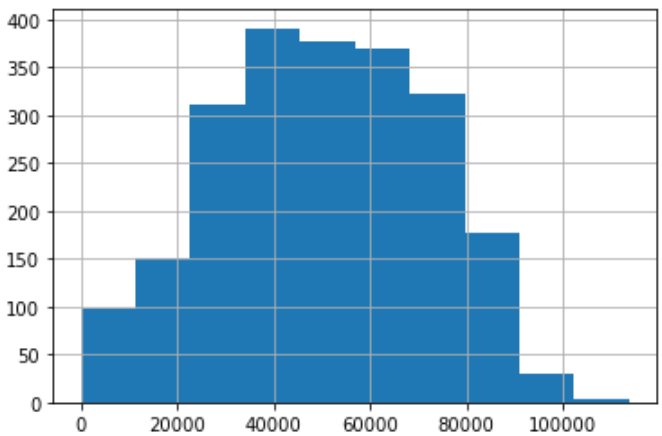
Out[34]:
```
(-15883.5, 117474.5)
```

In [35]:
```
data = data[data.Income<upper]
```

In [36]:
```
data.Income.hist()

# now the variable seems fine
```

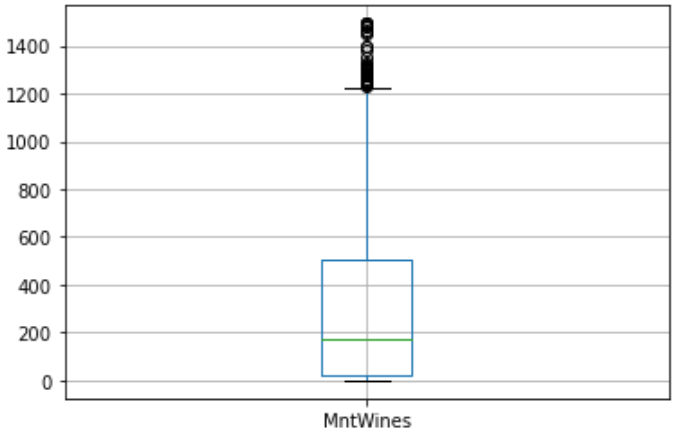
Out[36]:
```
<AxesSubplot:>
```



Amount Spent on Wines

In [37]:
```
data[['MntWines']].boxplot()
plt.show()

# we may eliminate
```



In [38]:
```
q1,q3 = data.MntWines.quantile((0.25,0.75))
iqr = q3-q1
lower = q1-1.5*iqr
upper = q3+1.5*iqr
lower,upper
```

Out[38]:
```
(-697.5, 1226.5)
```

In [39]:
```
# dropping outlier rows
data = data[data.MntWines<=upper]
```

Amount Spent on Sweet Products

In [40]:
```
q1,q3 = data.MntSweetProducts.quantile((0.25,0.75))
iqr = q3-q1
lower = q1-1.5*iqr
upper = q3+1.5*iqr
lower,upper
```

Out[40]:
```
(-45.5, 78.5)
```

In [41]:
```
data[data.MntSweetProducts>upper]
# seeing their Income the observations does not seem to be outliers, as these are high income customers
```

Out[41]:

ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	MntFruit
1826	1970	Graduation	Divorced	84835.000000	0	0	2014-06-16	0	189	10
2079	1947	2n Cycle	Married	81044.000000	0	0	2013-12-27	0	450	2
9264	1986	Graduation	Married	79529.000000	0	0	2014-04-27	1	423	4
2681	1984	2n Cycle	Married	65370.000000	0	0	2013-08-01	1	71	2
3767	1968	Graduation	Married	61314.000000	0	1	2013-04-25	1	378	
...	...	...	...	...	...	...	...	...	...	
5602	1989	PhD	Married	66973.000000	0	0	2013-05-17	98	466	2
4974	1970	Graduation	Single	83273.000000	1	2	2012-09-25	98	433	8
528	1978	Graduation	Married	65819.000000	0	0	2012-11-29	99	267	3
5255	1986	Graduation	Single	50029.020243	1	0	2013-02-20	19	5	
5250	1943	Master	Widow	54314.272727	0	0	2013-10-30	75	532	12

254 rows × 32 columns

Amount Spent on Meat Products

In [42]:
```
q1,q3 = data.MntMeatProducts.quantile((0.25,0.75))
iqr = q3-q1
lower = q1-1.5*iqr
```

```
upper = q3+1.5*iqr
lower,upper
```

Out[42]:
```
(-289.5, 522.5)
```

In [43]:
```
data[data.MntMeatProducts>=upper].Income.mean()
# so even these are high income customers, it won't be a good idea to drop these rows
```

Out[43]:
```
75510.80426623774
```

In [44]:
```
# we observe that amount spend variables are highly skewed and dropping the extreme observations is not a
```

Task-5 Encoding of Categorical Variables

In [45]:
```
data.select_dtypes('object')
# we remeber that we have already encoded Marital Status and Education Levels
# however, we need ordinal encoding of Education variable
```

Out[45]:

ID	Education	Marital_Status	Country	marital_education_labeler
1826	Graduation	Divorced	SP	20
1	Graduation	Single	CA	22
10476	Graduation	Married	US	21
1386	Graduation	Married	AUS	21
5371	Graduation	Single	SP	22
...	...	...	...	...
5798	Master	Married	GER	31
4345	2n Cycle	Single	AUS	02
5250	Master	Widow	AUS	33
7281	PhD	Single	AUS	42
1612	PhD	Single	AUS	42

2195 rows × 4 columns

In [46]:
```
# ordinal encoding of Education variable
data.Education.unique()
```

Out[46]:
```
array(['Graduation', 'PhD', '2n Cycle', 'Master', 'Basic'], dtype=object)
```

In [47]:
```
from sklearn.preprocessing import OrdinalEncoder
```

In [48]:
```
ord_encode = OrdinalEncoder(categories=[['Basic','2n Cycle','Graduation','Master','PhD']])
data['Education_label'] = ord_encode.fit_transform(data[['Education']]).astype('int')
```

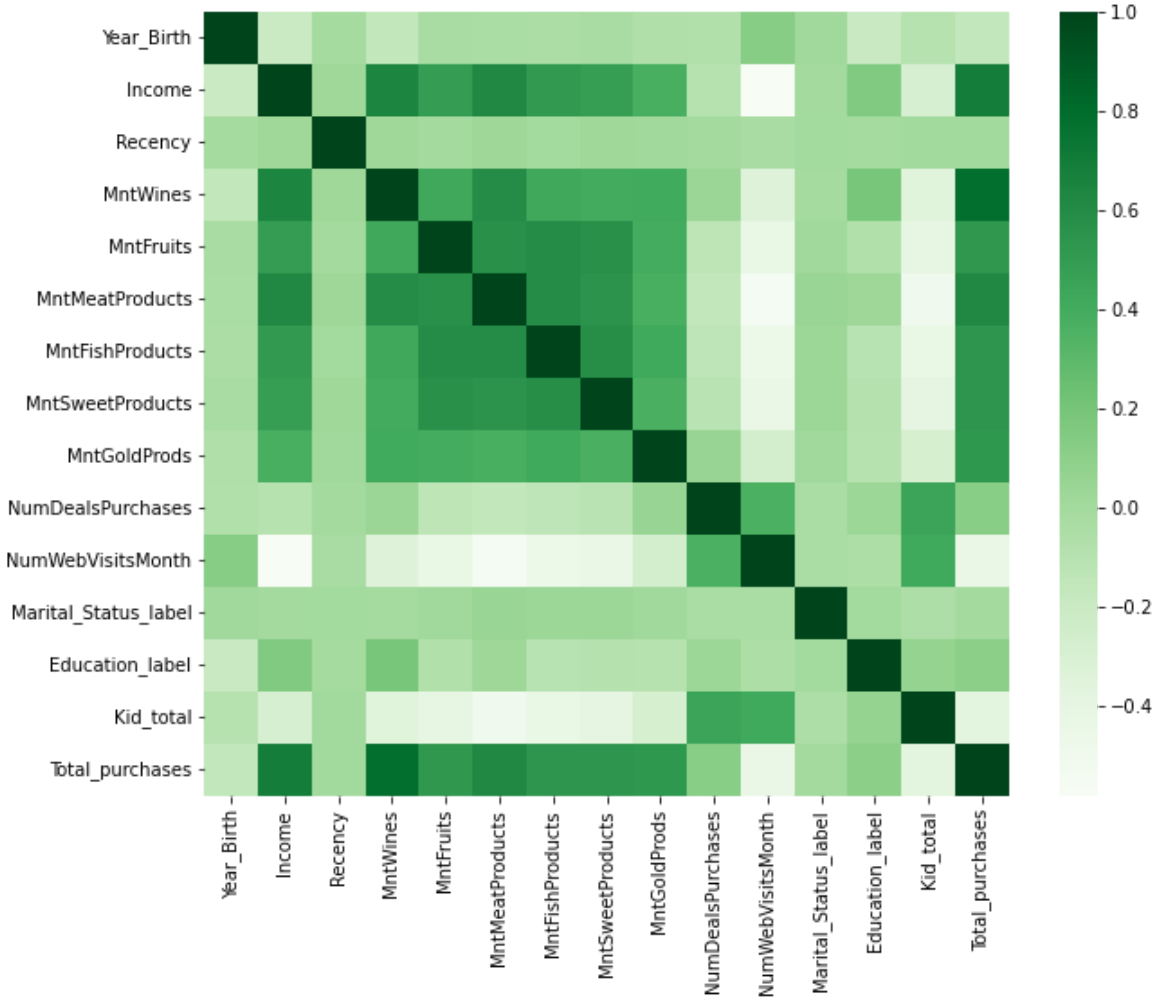
In [49]:
```
# we do not need marital_education_labeler, so we drop it
data.drop('marital_education_labeler',axis=1,inplace=True)
```

In [50]:
```
data.columns
```

Out[50]:
```
Index(['Year_Birth', 'Education', 'Marital_Status', 'Income', 'Kidhome',
       'Teenhome', 'Dt_Customer', 'Recency', 'MntWines', 'MntFruits',
       'MntMeatProducts', 'MntFishProducts', 'MntSweetProducts',
       'MntGoldProds', 'NumDealsPurchases', 'NumWebPurchases',
       'NumCatalogPurchases', 'NumStorePurchases', 'NumWebVisitsMonth',
       'AcceptedCmp3', 'AcceptedCmp4', 'AcceptedCmp5', 'AcceptedCmp1',
       'AcceptedCmp2', 'Response', 'Complain', 'Country',
       'Marital_Status_label', 'Education_label', 'Kid_total',
       'Total_purchases'],
      dtype='object')
```

Task-6 Heatmap to see pair-wise correlation

In [51]:
```
# heatmap of pair-wise correlation
plt.figure(figsize=(10,8))
sns.heatmap(data[['Year_Birth', 'Income', 'Recency', 'MntWines', 'MntFruits', 'MntMeatProducts',
       'MntFishProducts', 'MntSweetProducts', 'MntGoldProds',
       'NumDealsPurchases', 'NumWebVisitsMonth','Marital_Status_label',
       'Education_label', 'Kid_total', 'Total_purchases']].corr(),cmap='Greens')
plt.show()
```



Task-7 relation between age and shopping mode

In [52]:
```
import datetime
```

In [53]:
```
# creating agedescriberiable taking today as the current date
data['age'] = datetime.date.today().year-data.Year_Birth
```

In [54]:
```
# investigation age variable
data[data.age>90]
# here year of birth is wrong, as the difference between birth & registration in shop is more than 113 ye
# so let's drop these observations
```

Out[54]:

ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	MntFruits	...
11004	1893	2n Cycle	Single	60182.0	0	1	2014-05-17	23	8	0	...
1150	1899	PhD	Married	83532.0	0	0	2013-09-26	36	755	144	...
7829	1900	2n Cycle	Divorced	36640.0	1	0	2013-09-26	99	15	6	...

3 rows × 32 columns

In [55]:
```
data = data[data.age<=90]
```

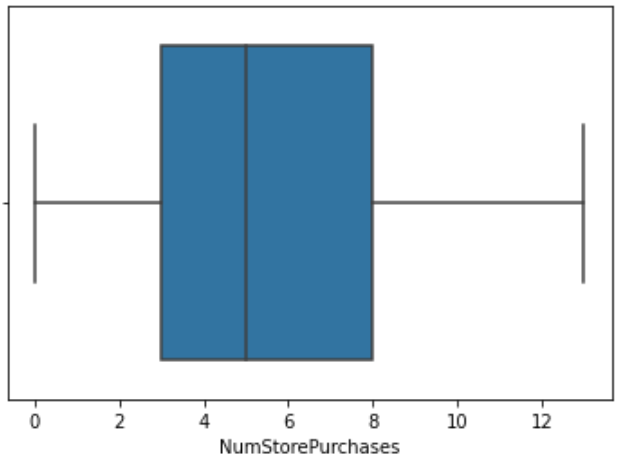
In [56]:
```
data.age.min(),data.age.max()
```

Out[56]: (27, 83)

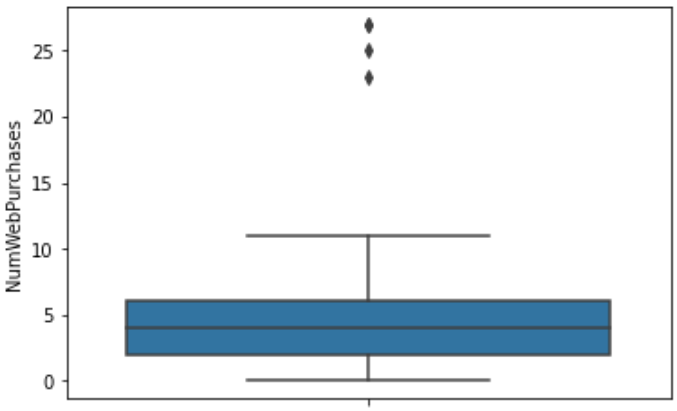
In [57]:
```
data['age_bins'] = pd.cut(x=data.age,bins=(20,30,40,50,60,70,80,90),labels=('20-30','30-40','40-50','50-6
```

In [58]:
```
# as we are going to do quantitive analysis on these variables, here outliers may affect
# so first we drop outliers
```

In [59]:
```
# cheking outliers in NumStorePurchases
sns.boxplot(data.NumStorePurchases)
plt.show()
```



In [60]:
```
# cheking outliers in NumWebPurchases
sns.boxplot(y=data.NumWebPurchases)
plt.show()
# so let's drop these outliers from dataset
```



In [61]:
```
q1,q3 = data.NumWebPurchases.quantile((0.25,0.75))
iqr = q3-q1
lower = q1-1.5*iqr
upper = q3+1.5*iqr
lower,upper
```

Out[61]: (-4.0, 12.0)

In [62]:
```
data = data[data.NumWebPurchases<=upper]
```

In [63]:
```
purchase_age_wise = data.pivot_table(index='age_bins',values=['NumStorePurchases','NumWebPurchases','NumC
purchase_age_wise['% store purchase'] = purchase_age_wise.NumStorePurchases/purchase_age_wise.Total_purch
purchase_age_wise['% web purchase'] = purchase_age_wise.NumWebPurchases/purchase_age_wise.Total_purchases
purchase_age_wise

# no clear cut pattern
# hypothesis doesn't seem to be true
```

Out[63]:

age_bins	NumCatalogPurchases	NumStorePurchases	NumWebPurchases	Total_purchases	% store purchase	% web purchase
20-30	63	91	51	205	44.390244	24.878049
30-40	598	1447	883	2928	49.419399	30.157104
40-50	1313	3265	2317	6895	47.353154	33.604061
50-60	1534	3509	2539	7582	46.280665	33.487207
60-70	1404	2889	2040	6333	45.618190	32.212222
70-80	739	1475	993	3207	45.993140	30.963517
80-90	1	4	3	8	50.000000	37.500000

Task-8 relation between number of kids and shopping mode

In [64]:
```
purchase_kids_wise = data.pivot_table(index='Kidhome',values=['NumStorePurchases','NumWebPurchases','NumC
purchase_kids_wise['% store purchase'] = purchase_kids_wise.NumStorePurchases/purchase_kids_wise.Total_pu
purchase_kids_wise['% web purchase'] = purchase_kids_wise.NumWebPurchases/purchase_kids_wise.Total_purcha
purchase_kids_wise
# no clear cut pattern
# hypothesis doesn't seem to be true
```

Out[64]:

Kidhome	NumCatalogPurchases	NumStorePurchases	NumWebPurchases	Total_purchases	% store purchase	% web purchase
0	4817	9066	6147	20030	45.262107	30.688967
1	801	3449	2545	6795	50.757910	37.454010
2	34	165	134	333	49.549550	40.240240

Task-9 Mode-wise Purchases

In [65]:
```
purchase_mode_wise = data.pivot_table(index='age_bins',margins=True,values=['NumStorePurchases','NumWebPu
purchase_mode_wise['% store purchases'] = 100*purchase_mode_wise['NumStorePurchases']/purchase_mode_wise[
purchase_mode_wise['% web purchases'] = 100*purchase_mode_wise['NumWebPurchases']/purchase_mode_wise['Tot
purchase_mode_wise['% catalogue purchases'] = 100*purchase_mode_wise['NumCatalogPurchases']/purchase_mode
purchase_mode_wise.T['All']

# store purchase is still the dominant medium
```

Out[65]:
```
NumCatalogPurchases       5652.000000
NumStorePurchases        12680.000000
NumWebPurchases           8826.000000
Total_purchases          27158.000000
% store purchases           46.689742
% web purchases             32.498711
% catalogue purchases       20.811547
Name: All, dtype: float64
```

Task-10 Country-wise Total Purchases

In [66]:
```
data.pivot_table(index='Country',values='Total_purchases',aggfunc='mean').sort_values(by='Total_purchases
# clearly US does not fare better than rest of the world
```

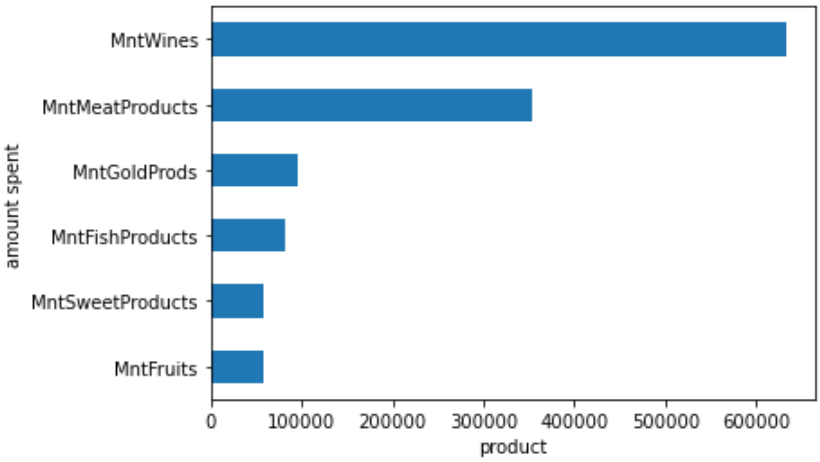
Out[66]:

Country	Total_purchases
ME	17.333333
US	13.370370
GER	12.714286
CA	12.698842
SA	12.631098
SP	12.294393
AUS	11.993590
IND	11.662069

Task-11 best and least performing products

In [115…

```
data[['MntWines', 'MntFruits',
       'MntMeatProducts', 'MntFishProducts', 'MntSweetProducts',
       'MntGoldProds']].T.sum(axis=1).sort_values().plot(kind='barh')
plt.xlabel('product')
plt.ylabel('amount spent')
plt.show()
```

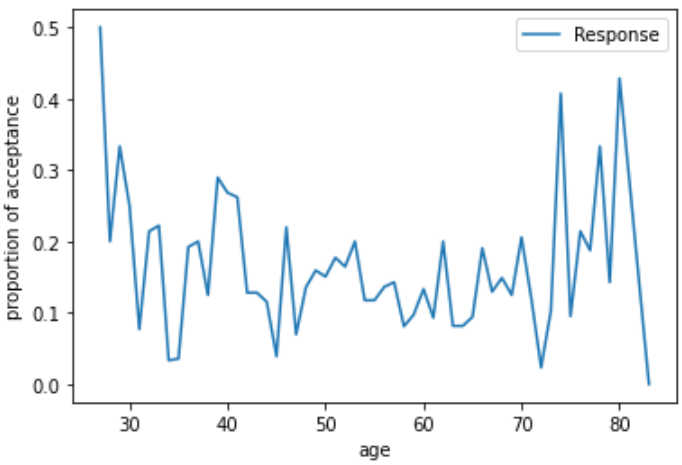


Task-12 Pattern between age and last campaign acceptance rate

In [115…

```
data.pivot_table(index='age',values='Response',aggfunc='mean').plot()
plt.xlabel('age')
plt.ylabel('proportion of acceptance')
plt.show()

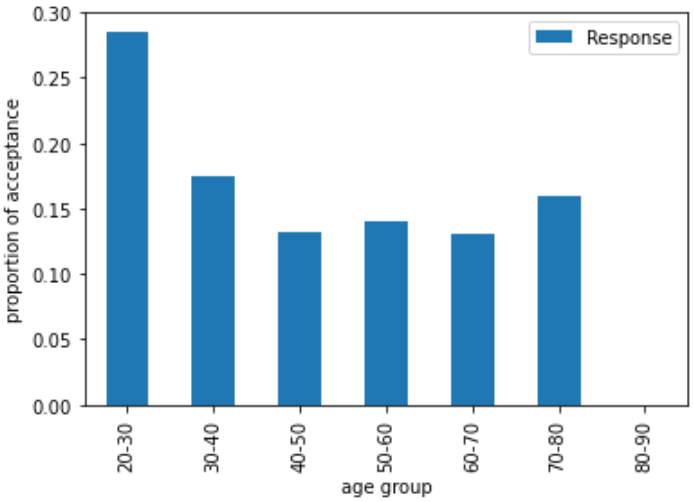
# Below 30 and above 70 aged customers have slightly higher acceptance rate
```



In [115…

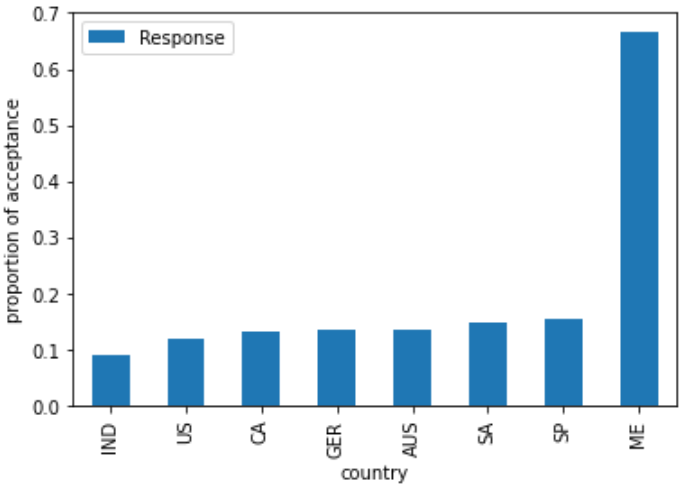
```
data.pivot_table(index='age_bins',values='Response',aggfunc='mean').plot(kind='bar')
plt.xlabel('age group')
plt.ylabel('proportion of acceptance')
```

```
plt.show()
# again, 20-40 age group and 70-80 age group have slightly higher acceptance rate then mid-aged customers
```



Task-13 country-wise last campaign acceptance rate

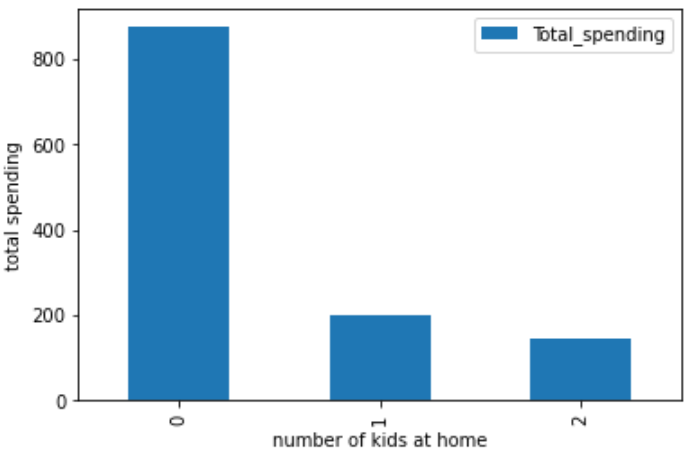
In [115…
```
data.pivot_table(index='Country',values='Response',aggfunc='mean').sort_values(by='Response').plot(kind='
plt.xlabel('country')
plt.ylabel('proportion of acceptance')
plt.show()
```



Task-14 relation between number of kids at home and amount spent

In [115…
```
data['Total_spending'] = data.MntFishProducts+data.MntFruits+data.MntGoldProds+data.MntMeatProducts+data.
```

In [115…
```
data.pivot_table(index='Kidhome',values='Total_spending',aggfunc='mean').sort_values(by='Total_spending',
plt.xlabel('number of kids at home')
plt.ylabel('total spending')
plt.show()
```



In [116…
```
np.corrcoef(data.Kidhome,data.Total_spending)
```

Out[1160]:
```
array([[ 1.        , -0.55786997],
       [-0.55786997,  1.        ]])
```

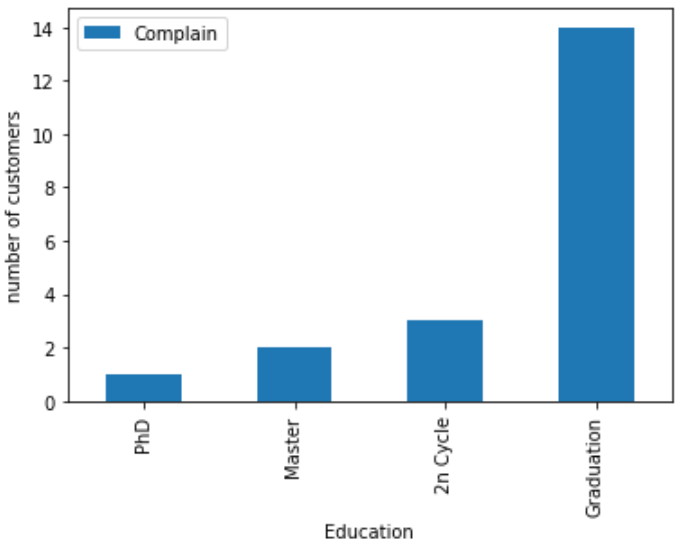
In [116…
```
data.pivot_table(index='Kidhome',values='Total_spending',aggfunc='mean')
```

Out[1161]:

Kidhome	Total_spending
0	874.577724
1	201.891256
2	145.250000

Task-15 Education background of customers who complained in the last 2 years

In [116…
```
data[data.Complain==1].pivot_table(index='Education',values='Complain',aggfunc='sum').sort_values(by='Com
plt.ylabel('number of customers')
plt.show()
# so complainants were primarily graduate students
```



Completed